Università di Genova

Embedded Systems

Interrupts

Prof. Enrico Simetti

Interrupts

Interrupt: an asynchronous signal from hardware indicating the need for attention (hardware interrupt) or a synchronous event in software indicating the need for a change in execution (software interrupt or software trap)



Synchronous Interrupts (Traps)

Synchronous and asynchronous with respect to what?

To program execution!

Synchronous means that the interrupt depends on the instruction we are currently executing (e.g. a divide by zero trap)

```
while (1) {
    int a = 2;
    double result = 100;
    for (i = 0; i <= 2; i++, a--) {
      result = result / a;
}
```

Synchronous Interrupts (Traps)

a	2
result	100

```
while (1) {
  int a = 2;
  double result = 100;
  for (i = 0; i <= 2; i++, a--) {
    result = result / a;
}
```

Synchronous Interrupts (Traps)

a	2
result	50

```
while (1) {
  int a = 2;
  double result = 100;
  for (i = 0; i <= 2; i++, a--) {
    result = result / a;
}
```



Synchronous Interrupts (Traps)

a	1
result	50

```
while (1) {
  int a = 2;
  double result = 100;
  for (i = 0; i <= 2; i++, a--) {
    result = result / a;
}
```

Synchronous Interrupts (Traps)

a	1
result	50

```
while (1) {
  int a = 2;
  double result = 100;
  for (i = 0; i <= 2; i++, a--) {
    result = result / a;
}
```



Synchronous Interrupts (Traps)

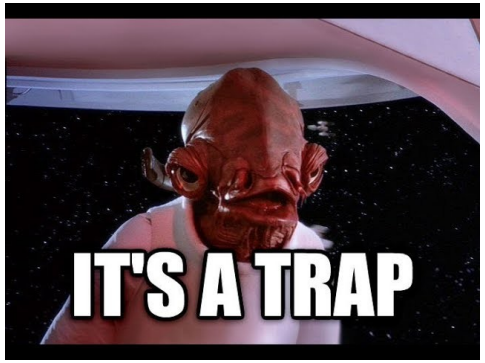
a	0
result	50

```
while (1) {
   int a = 2;
   double result = 100;
   for (i = 0; i <= 2; i++, a--) {
     result = result / a;
}
```

Synchronous Interrupts (Traps)

a	0
result	???

```
while (1) {
   int a = 2;
   double result = 100;
   for (i = 0; i <= 2; i++, a--) {
      result = result / a;
}
```



Asynchronous Interrupts

Asynchronous means that it does have any temporal relation to which part of the code the microcontroller is executing (the timer
expires independently of which point the program has currently reached)

Interrupt Service Routine

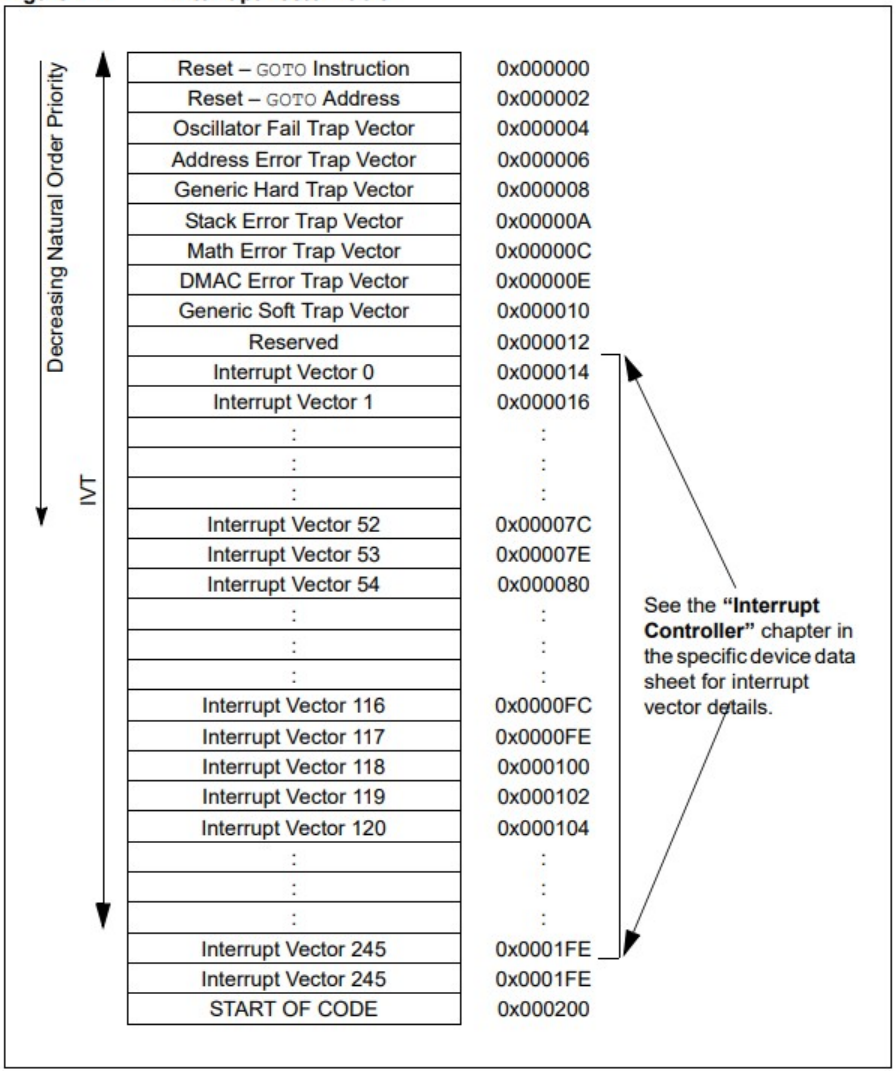
An ISR is like any other C function with some special rules:

- each ISR has a special name which is used for linking its code to the IVT
- no parameters can be passed to the ISR
- no parameters can be returned by an ISR (void return type is mandatory)
- the flag generating the interrupt should be generally cleared in the ISR
- ISR code should not contain blocking sections
- ISRs should not be called by main line code
- Care must be taken with shared data

```
void __attribute__((__interrupt__, __auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  [...] // do some action
}
```

Interrupt Vector Table

Figure 1-1: Interrupt Vector Table



ISR Names

ISR names for dsPIC33 can be found in the dsPIC C compiler document, page 11

This is just a snippet!

TABLE 8-4: INTERRUPT VECTORS - dsPIC33F DSCs/PIC24H MCUs

IRQ#	Primary Name	Alternate Name	Vector Function
N/A	_StackError	_AltStackError	Stack error trap
N/A	_MathError	_AltMathError	Math error trap
N/A	_DMACError	_AltDMACError	DMA conflict error trap
N/A	_ReservedTrap6	_AltReservedTrap6	Reserved
N/A	_ReservedTrap7	_AltReservedTrap7	Reserved
0	_INT0Interrupt	_AltINT0Interrupt	INT0 External interrupt 0
1	_IC1Interrupt	_AltIC1Interrupt	IC1 Input capture 1
2	_OC1Interrupt	_AltOC1Interrupt	OC1 Output compare 1
3	_T1Interrupt	_AltT1Interrupt	TMR1 Timer 1 expired
4	_DMA0Interrupt	_AltDMA0Interrupt	DMA 0 interrupt
5	_IC2Interrupt	_AltIC2Interrupt	IC2 Input capture 2
6	_OC2Interrupt	_AltOC2Interrupt	OC2 Output compare 2
7	_T2Interrupt	_AltT2Interrupt	TMR2 Timer 2 expired
8	_T3Interrupt	_AltT3Interrupt	TMR3 Timer 3 expired
9	_SPI1ErrInterrupt	_AltSPI1ErrInterrupt	SPI1 error interrupt
10	_SPI1Interrupt	_AltSPI1Interrupt	SPI1 transfer completed interrupt
11	_U1RXInterrupt	_AltU1RXInterrupt	UART1RX Uart 1 Receiver
12	_U1TXInterrupt	_AltU1TXInterrupt	UART1TX Uart 1 Transmitter
13	_ADC1Interrupt	_AltADC1Interrupt	ADC 1 convert completed
14	_DMA1Interrupt	_AltDMA1Interrupt	DMA 1 interrupt
15	_Interrupt15	_AltInterrupt15	Reserved

Interrupts: How They Work

- All the peripherals have one or more interrupt flags which are set by hardware when a special event occurs
- All interrupt flags are sampled at the beginning of each instruction cycle
- If there is a pending request and if the corresponding interrupt generation is enabled an interrupt will be presented to the processor.
- The processor then saves the following information on the software stack (interrupt context switch):
  - the current PC value
  - the Processor Status register
- The CPU sets its priority level equal to the one of the pending interrupt
- The Interrupt Service Routine corresponding to the event is executed
- After the ISR, the previous context is recovered, and the program execution continues

Interrupt Behaviour

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  [...] // do some action
}
```

```
IEC0bits.INT0IE = 1;
while (1) {
   int a = 2;
   double result = 100;
   for (i = 0; i <= 2; i++, a--) {
     result = result / a;
}
```

INT0IE	1
INT0IF	0
CPU Priority	0

Interrupt Behaviour

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  [...] // do some action
}
```

```
IEC0bits.INT0IE = 1;
while (1) {
    int a = 2;
    double result = 100;
    for (i = 0; i <= 2; i++, a--) {
      result = result / a;
}
```

INT0IE	1
INT0IF	0
CPU Priority	0

Interrupt Behaviour

```
IEC0bits.INT0IE = 1;
while (1) {
    int a = 2;
    double result = 100;
    for (i = 0; i <= 2; i++, a--)
      result = result / a;
}
```

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  [...] // do some action
}
```

INTERRUPT RAISED



INT0IE	1
INT0IF	1
CPU Priority	0

Interrupt Behaviour

```
IEC0bits.INT0IE = 1;
while (1) {
    int a = 2;
    double result = 100;
    for (i = 0; i <= 2; i++, a--)
      result = result / a;
}
```

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  [...] // do some action
}
```

INTERRUPT



INT0IE	1
INT0IF	1
CPU Priority	4

Interrupt Behaviour

```
IEC0bits.INT0IE = 1;
while (1) {
    int a = 2;
    double result = 100;
    for (i = 0; i <= 2; i++, a--)
      result = result / a;
}
```

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  [...] // do some action
}
```

INTERRUPT



INT0IE	1
INT0IF	1
CPU Priority	4

Interrupt Behaviour

```
IEC0bits.INT0IE = 1;
while (1) {
    int a = 2;
    double result = 100;
    for (i = 0; i <= 2; i++, a--)
      result = result / a;
}
```

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  [...] // do some action
}
```

INTERRUPT



INT0IE	1
INT0IF	0
CPU Priority	4

Interrupt Behaviour

```
IEC0bits.INT0IE = 1;
while (1) {
    int a = 2;
    double result = 100;
    for (i = 0; i <= 2; i++, a--)
      result = result / a;
}
```

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  [...] // do some action
}
```

INTERRUPT COMPLETED



INT0IE	1
INT0IF	0
CPU Priority	0

Interrupt Behaviour

```
IEC0bits.INT0IE = 1;
while (1) {
    int a = 2;
    double result = 100;
    for (i = 0; i <= 2; i++, a--)
      result = result / a;
}
```

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  [...] // do some action
}
```

INT0IE	1
INT0IF	0
CPU Priority	0

Nested Interrupts

An interrupt with a strictly higher priority can interrupt another interrupt -> nested interrupts.

The CPU will increase its priority to the one of the higher priority interrupt, and will resume the first interrupt at the end, unless another higher priority interrupts occurs.

Nested Interrupts

An interrupt with equal or lower priority does not interrupt the on-going one. If the conditions triggering the interrupt are true at the end of the on-going one, then it will be executed.

Notice that the interrupt is thus delayed.

Even if during this delay the conditions triggering the second interrupt are repeated, you will get a single interrupt once the higher priority finishes.

Interrupt Flag Status Registers

Register 6-5: IFS0: Interrupt Flag Status Register 0

Upper Byte:							
R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
CNIF	MI2CIF	SI2CIF	NVMIF	ADIF	U1TXIF	U1RXIF	SPI1IF
bit 15							bit 8

Lower Byte:							
R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
T3IF	T2IF	OC2IF	IC2IF	T1IF	OC1IF	IC1IF	INT0IF
bit 7							bit 0

bit 15 **CNIF:** Input Change Notification Flag Status bit
1 = Interrupt request has occurred
0 = Interrupt request has not occurred

bit 14 **MI2CIF:** I^2C Bus Collision Flag Status bit
1 = Interrupt request has occurred
0 = Interrupt request has not occurred

IFSx: Interrupt Flag Status registers, contains the flags that notify th program of certain events. For example IFS0bits.T1IF tells if the Timer 1 has expired. These flags are set to ’1’ automatically, but must be set to ’0’ by the user.
Page 58 of the datasheet shows the location of all the IFs

Interrupt Enable Control Registers

Register 6-8: IEC0: Interrupt Enable Control Register 0

Upper Byte:							
R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
CNIE	MI2CIE	SI2CIE	NVMIE	ADIE	U1TXIE	U1RXIE	SPI1IE
bit 15							bit 8

Lower Byte:							
R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
T3IE	T2IE	OC2IE	IC2IE	T1IE	OC1IE	IC1IE	INT0IE
bit 7							bit 0

bit 15 **CNIE:** Input Change Notification Interrupt Enable bit
1 = Interrupt request enabled
0 = Interrupt request not enabled

bit 14 **MI2CIE:** I^2C Bus Collision Interrupt Enable bit
1 = Interrupt request enabled
0 = Interrupt request not enabled

IECx: Interrupt Enable Control example, it is used to enable an interrupt. For example IEC0bits.T1IE = 1 tells that we want an interrupt every time that IFS0bits.T1IF becomes '1'.

see Chapter 6, pag. 148 of the reference manual

Interrupts and Shared Variables

```
int a;
int main() {
…
IEC0bits.INT0IE = 1;
while (1) {
    a = 0, b = 1;
    double result = 0;
    for (i = 0; i <= 2; i++) {
      a = a + 1;
      b = b * a;
      result = b + 2;
}
```

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  a = 0;
  [...] // do some action
}
```

a	0
b	1
result	0
INT0IF	0
INT0IE	1

Interrupts and Shared Variables

```
int a;
int main() {
…
IEC0bits.INT0IE = 1;
while (1) {
    a = 0, b = 1;
    double result = 0;
    for (i = 0; i <= 2; i++) {
      a = a + 1;
      b = b * a;
      result = b + 2;
}
```

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  a = 0;
  [...] // do some action
}
```

a	0
b	1
result	0
INT0IF	0
INT0IE	1

R0	
R1	
ACC	

Interrupts and Shared Variables

```
int a;
int main() {
…
IEC0bits.INT0IE = 1;
while (1) {
    a = 0, b = 1;
    double result = 0;
    for (i = 0; i <= 2; i++) {
      a = a + 1;
      b = b * a;
      result = b + 2;
}
```

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  a = 0;
  [...] // do some action
}
```

Read from memory

a	0
b	1
result	0
INT0IF	0
INT0IE	1

R0	0
R1	X
ACC	X

Interrupts and Shared Variables

```
int a;
int main() {
…
IEC0bits.INT0IE = 1;
while (1) {
    a = 0, b = 1;
    double result = 0;
    for (i = 0; i <= 2; i++) {
      a = a + 1;
      b = b * a;
      result = b + 2;
}
```

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  a = 0;
  [...] // do some action
}
```

a	0
b	1
result	0
INT0IF	0
INT0IE	1

R0	0
R1	1
ACC	X

Interrupts and Shared Variables

```
int a;
int main() {
…
IEC0bits.INT0IE = 1;
while (1) {
    a = 0, b = 1;
    double result = 0;
    for (i = 0; i <= 2; i++) {
      a = a + 1;
      b = b * a;
      result = b + 2;
}
```

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  a = 0;
  [...] // do some action
}
```

a	0
b	1
result	0
INT0IF	0
INT0IE	1

R0	0
R1	1
ACC	1

Interrupts and Shared Variables

```
int a;
int main() {
…
IEC0bits.INT0IE = 1;
while (1) {
    a = 0, b = 1;
    double result = 0;
    for (i = 0; i <= 2; i++) {
      a = a + 1;
      b = b * a;
      result = b + 2;
}
```

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  a = 0;
  [...] // do some action
}
```

From the CPU accumulator to the memory

a	1
b	1
result	0
INT0IF	0
INT0IE	1

R0	0
R1	1
ACC	1

Interrupts and Shared Variables

```
int a;
int main() {
…
IEC0bits.INT0IE = 1;
while (1) {
    a = 0, b = 1;
    double result = 0;
    for (i = 0; i <= 2; i++) {
      a = a + 1;
      b = b * a;
      result = b + 2;
}
```

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  a = 0;
  [...] // do some action
}
```

a	1
b	1
result	3
INT0IF	0
INT0IE	1

Interrupts and Shared Variables

```
int a;
int main() {
…
IEC0bits.INT0IE = 1;
while (1) {
    a = 0, b = 1;
    double result = 0;
    for (i = 0; i <= 2; i++) {
      a = a + 1;
      b = b * a;
      result = b + 2;
}
```

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  a = 0;
  [...] // do some action
}
```

a	1
b	1
result	3
INT0IF	0
INT0IE	1

R0	X
R1	X
ACC	X

Interrupts and Shared Variables

```
int a;
int main() {
…
IEC0bits.INT0IE = 1;
while (1) {
    a = 0, b = 1;
    double result = 0;
    for (i = 0; i <= 2; i++) {
      a = a + 1;
      b = b * a;
      result = b + 2;
}
```

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  a = 0;
  [...] // do some action
}
```

Read from memory

a	1
b	1
result	3
INT0IF	0
INT0IE	1

R0	1
R1	1
ACC	X

Interrupts and Shared Variables

```
int a;
int main() {
…
IEC0bits.INT0IE = 1;
while (1) {
    a = 0, b = 1;
    double result = 0;
    for (i = 0; i <= 2; i++) {
      a = a + 1;
      b = b * a;
      result = b + 2;
}
```

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  a = 0;
  [...] // do some action
}
```

Interrupt!

a	1
b	1
result	3
INT0IF	1
INT0IE	1

The registers are pushed and saved

R0	1
R1	1
ACC	X

Interrupts and Shared Variables

```
int a;
int main() {
…
IEC0bits.INT0IE = 1;
while (1) {
    a = 0, b = 1;
    double result = 0;
    for (i = 0; i <= 2; i++) {
      a = a + 1;
      b = b * a;
      result = b + 2;
}
```

Interrupt!

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  a = 0;
  [...] // do some action
}
```

a	1
b	1
result	3
INT0IF	1
INT0IE	1

Interrupts and Shared Variables

```
int a;
int main() {
…
IEC0bits.INT0IE = 1;
while (1) {
    a = 0, b = 1;
    double result = 0;
    for (i = 0; i <= 2; i++) {
      a = a + 1;
      b = b * a;
      result = b + 2;
}
```



```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  a = 0;
  [...] // do some action
}
```

a	1
b	1
result	3
INT0IF	1
INT0IE	1

Interrupts and Shared Variables

```
int a;
int main() {
…
IEC0bits.INT0IE = 1;
while (1) {
    a = 0, b = 1;
    double result = 0;
    for (i = 0; i <= 2; i++) {
      a = a + 1;
      b = b * a;
      result = b + 2;
}
```



```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  a = 0;
  [...] // do some action
}
```

a	1
b	1
result	3
INT0IF	0
INT0IE	1

Interrupts and Shared Variables

```
int a;
int main() {
…
IEC0bits.INT0IE = 1;
while (1) {
    a = 0, b = 1;
    double result = 0;
    for (i = 0; i <= 2; i++) {
      a = a + 1;
      b = b * a;
      result = b + 2;
}
```



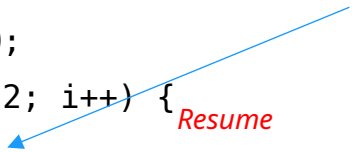
```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  a = 0;
  [...] // do some action
}
```

a	0
b	1
result	3
INT0IF	0
INT0IE	1

Interrupts and Shared Variables

```
int a;
int main() {
…
IEC0bits.INT0IE = 1;
while (1) {
    a = 0, b = 1;
    double result = 0;
    for (i = 0; i <= 2; i++) {
      a = a + 1;
      b = b * a;
      result = b + 2;
}
```

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  a = 0;
  [...] // do some action
}
```



a	0
b	1
result	3
INT0IF	0
INT0IE	1

Interrupts and Shared Variables

```
int a;
int main() {
…
IEC0bits.INT0IE = 1;
while (1) {
    a = 0, b = 1;
    double result = 0;
    for (i = 0; i <= 2; i++) {
      a = a + 1;
      b = b * a;
      result = b + 2;
}
```

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  a = 0;
  [...] // do some action
}
```

The registers are pulled and restored

a	0
b	1
result	3
INT0IF	0
INT0IE	1

R0	1
R1	1
ACC	X

Interrupts and Shared Variables

```
int a;
int main() {
…
IEC0bits.INT0IE = 1;
while (1) {
    a = 0, b = 1;
    double result = 0;
    for (i = 0; i <= 2; i++) {
      a = a + 1;
      b = b * a;
      result = b + 2;
}
```

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  a = 0;
  [...] // do some action
}
```

a	0
b	1
result	3
INT0IF	0
INT0IE	1

R0	1
R1	1
ACC	2

Interrupts and Shared Variables

```
int a;
int main() {
…
IEC0bits.INT0IE = 1;
while (1) {
    a = 0, b = 1;
    double result = 0;
    for (i = 0; i <= 2; i++) {
      a = a + 1;
      b = b * a;
      result = b + 2;
}
```

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  a = 0;
  [...] // do some action
}
```

From the CPU accumulator to the memory

a	2
b	1
result	3
INT0IF	0
INT0IE	1

R0	1
R1	1
ACC	2

Interrupts and Shared Variables

```
int a;
int main() {
…
IEC0bits.INT0IE = 1;
while (1) {
    a = 0, b = 1;
    double result = 0;
    for (i = 0; i <= 2; i++) {
      a = a + 1;
      b = b * a;
      result = b + 2;
}
```

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  a = 0;
  [...] // do some action
}
```

a	2
b	1
result	3
INT0IF	0
INT0IE	1

Interrupts and Shared Variables

```
int a;
int main() {
…
IEC0bits.INT0IE = 1;
while (1) {
    a = 0, b = 1;
    double result = 0;
    for (i = 0; i <= 2; i++) {
      a = a + 1;
      b = b * a;
      result = b + 2;
}
```

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  a = 0;
  [...] // do some action
}
```

a	2
b	2
result	3
INT0IF	0
INT0IE	1

Interrupts and Shared Variables

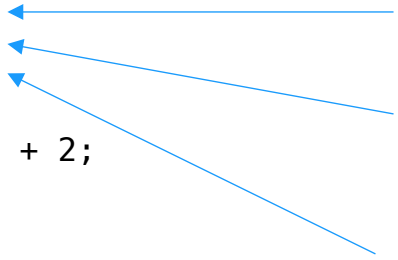
```
int a;
int main() {
…
IEC0bits.INT0IE = 1;
while (1) {
    a = 0, b = 1;
    double result = 0;
    for (i = 0; i <= 2; i++) {
      a = a + 1;
      b = b * a;
      result = b + 2;
}
```

```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  a = 0;
  [...] // do some action
}
```

a	2
b	2
result	4
INT0IF	0
INT0IE	1

Interrupts and Shared Variables

```
int a;
int main() {
…
IEC0bits.INT0IE = 1;
while (1) {
    a = 0, b = 1;
    double result = 0;
    for (i = 0; i <= 2; i++) {
      a = a + 1;
      b = b * a;
      result = b + 2;
}
```



```
void __attribute__((__interrupt__,
__auto_psv__)) _INT0Interrupt() {
  IFS0bits.INT0IF = 0; // reset interrupt flag
  a = 0;
  [...] // do some action
}
```

Knowing that b = 1, if the interrupt happens:

- Before the statement, a will be zeroed then incremented
Result = 3

- During the statement, after a is read from the memory
Result = 4

- After the statement, a is zeroed by the interrupt
Result = 2

How to Protect Shared Variables?

How to prevent such issues?

What if I want to execute my algorithm on a consistent set of data?

Critical Regions

```
int a;
int main() {
…
IEC0bits.INT0IE = 1;
while (1) {
    a = 0, b = 1;
    double result = 0;
    IEC0bits.INT0IE = 0;
    for (i = 0; i <= 2; i++) {
      a = a + 1;
      b = b * a;
      result = b + 2;
        }
        IEC0bits.INT0IE = 1;
```



Critical region
The interrupt is disabled and does not interfere

Critical Regions

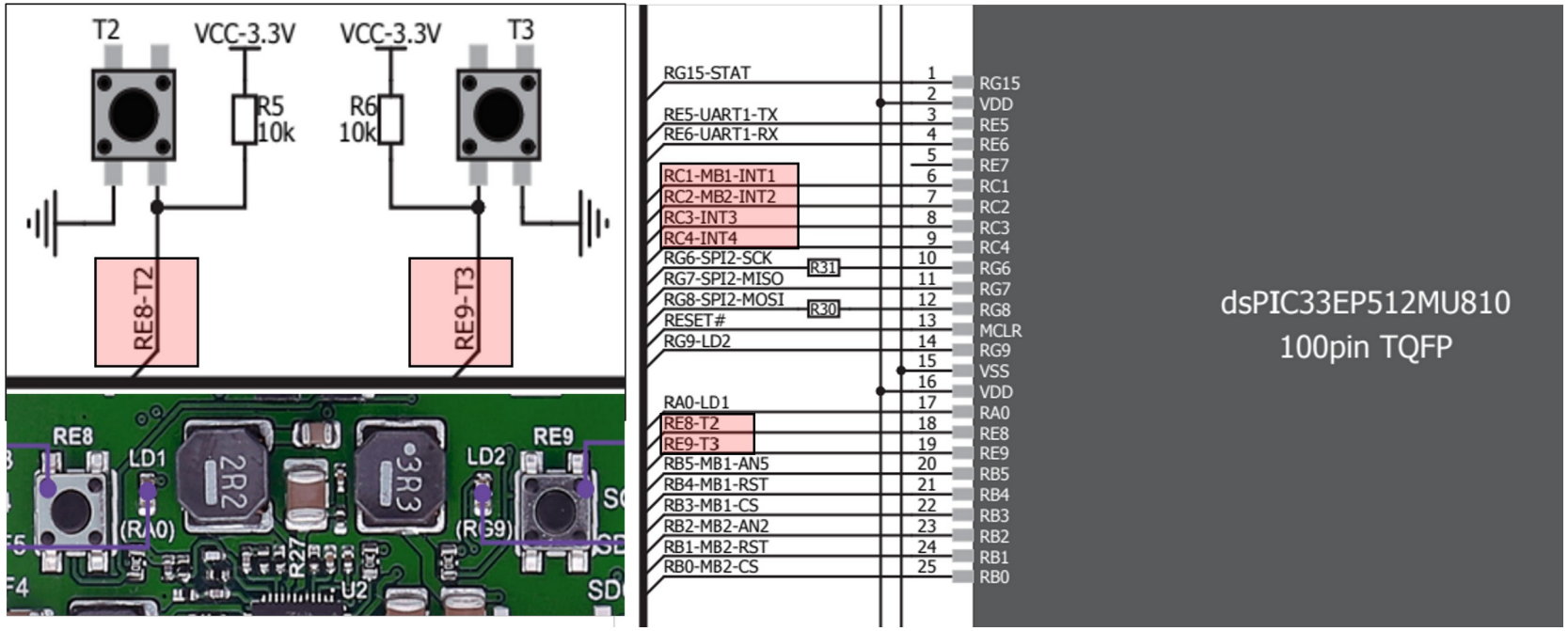
The interrupt does not fire inside since the IE is 0

If the IF stays 1, then as soon as IE goes back to 1 the interrupt will fire (delayed from its original event)

If multiple events have occurred, which would have made the IF go to 1, they are lost. Only one interrupt will occur when IE goes back to 1.

Peripheral Pin Select

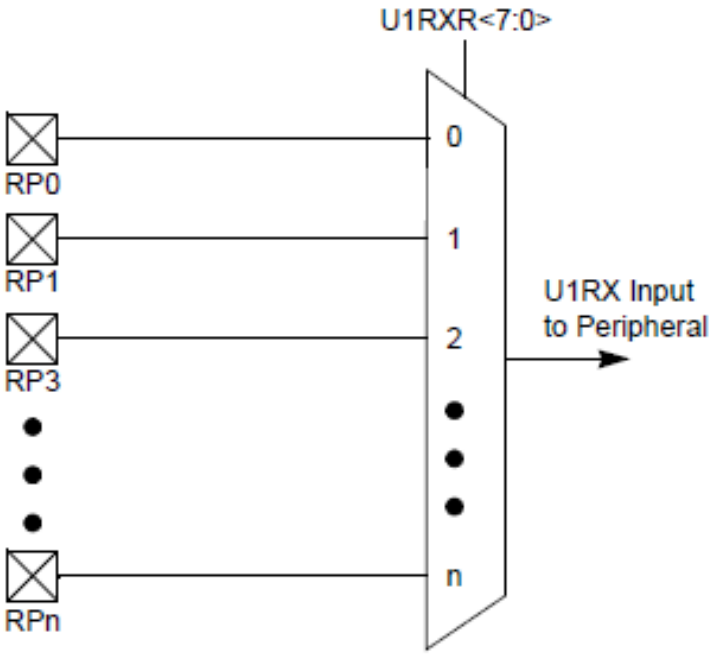
Unlike other devices, here you need to assign pins with the Peripheral Pin Select.



E.g. assign External Interrupt 1 (INT1) to RE8 Pin (left button)

Peripheral Pin Select (Input)

Figure 10-4: Remappable Input for U1RX



Note: For input only, Peripheral Pin Select functionality does not have priority over TRISx settings. Therefore, when configuring the RPn pin for an input, the corresponding bit in the TRISx register must be configured for input (set to '1').

Peripheral Pin Select Registers (Input)

The following table shows input mapping resources:

Table 10-1: Selectable Input Sources (Maps Input to Function)

Input Name[1]	Function Name	Register	Configuration Bits
External Interrupt 1	INT1	RPINR0	INT1R<7:0>
External Interrupt 2	INT2	RPINR1	INT2R<7:0>
External Interrupt 3	INT3	RPINR1	INT3R<7:0>
External Interrupt 4	INT4	RPINR2	INT4R<7:0>
Timer1 External Clock	T1CK	RPINR2	T1CKR<7:0>
Timer2 External Clock	T2CK	RPINR3	T2CKR<7:0>
Timer3 External Clock	T3CK	RPINR3	T3CKR<7:0>
Timer4 External Clock	T4CK	RPINR4	T4CKR<7:0>
Timer5 External Clock	T5CK	RPINR5	T5CKR<7:0>
Timer6 External Clock	T6CK	RPINR6	T6CKR<7:0>
Timer7 External Clock	T7CK	RPINR7	T7CKR<7:0>
Timer8 External Clock	T8CK	RPINR8	T8CKR<7:0>
Input Capture 1	IC1	RPINR7	IC1R<7:0>
Input Capture 2	IC2	RPINR7	IC2R<7:0>
Input Capture 3	IC3	RPINR8	IC3R<7:0>
Input Capture 4	IC4	RPINR8	IC4R<7:0>
Input Capture5	IC5	RPINR9	IC5R<7:0>
Input Capture 6	IC6	RPINR9	IC6R<7:0>
Input Capture 7	IC7	RPINR10	IC7R<7:0>
Input Capture 8	IC8	RPINR10	IC8R<7:0>
Output Compare Fault A	OCFA	RPINR11	OCFAR<7:0>
Output Compare Fault B	OCFB	RPINR11	OCFBR<7:0>
PWM Fault 1	FLT1	RPINR12	FLT1R<7:0>
PWM Fault 2	FLT2	RPINR12	FLT2R<7:0>
PWM Fault 3	FLT3	RPINR13	FLT3R<7:0>
PWM Fault 4	FLT4	RPINR13	FLT4R<7:0>
QEI1 Phase A	QEA1	RPINR14	QEA1R<7:0>

Mapping Inputs

The thing to remember is

Functionality = Remappable Input Pin

So, for example for U1RX

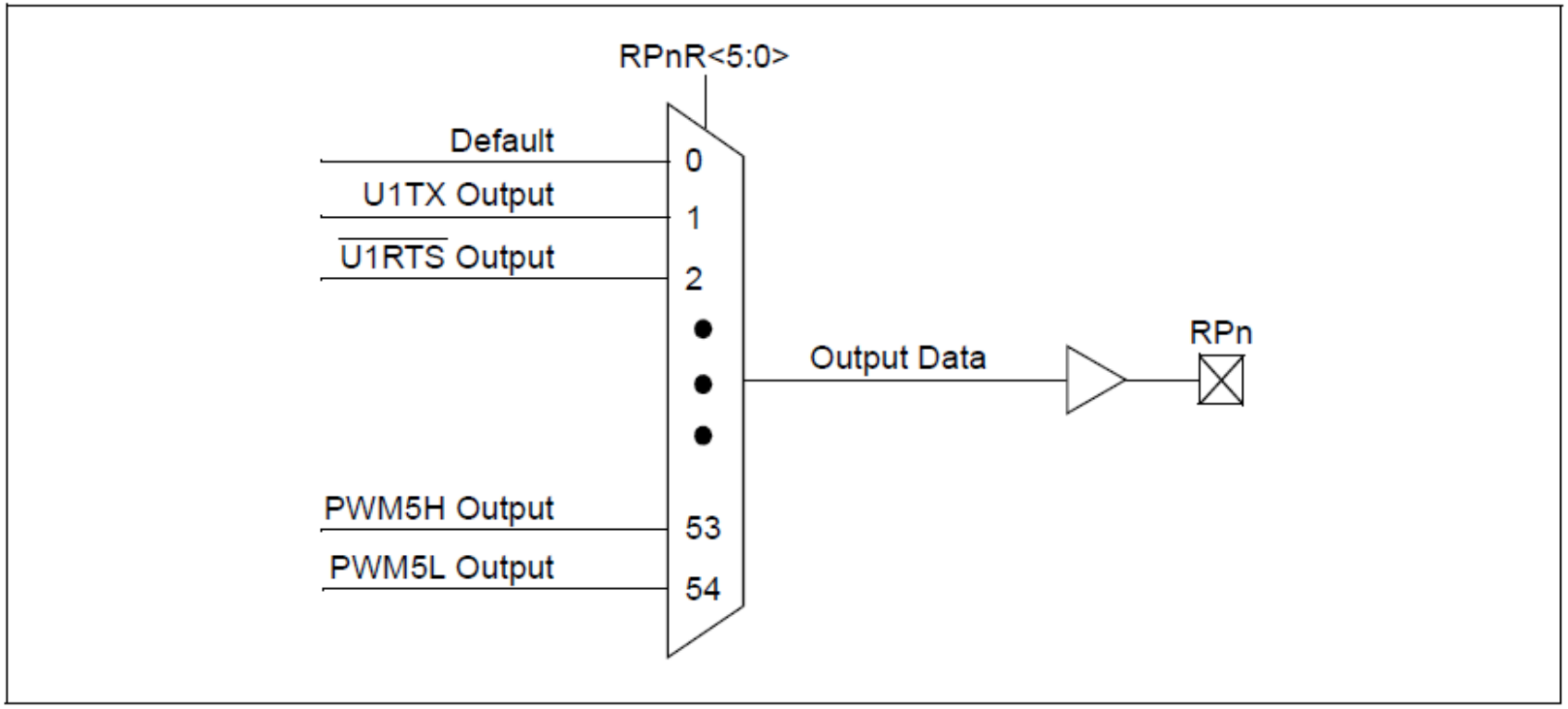
```
TRISAbits.TRISA4 = 1; // RPI20 is
RA4

RPINR18bits.U1RXR = 20;
```

maps the U1RX to RPI20.

Peripheral Pin Select (Output)

Figure 10-5: Multiplexing of Remappable Outputs for RPn



Peripheral Pin Select Values (Output)

The following table shows some output mapping resources:

Table 10-2: Output Selection for Remappable Pins (RPn)

Function	RPnR<5:0>	Output Name
DEFAULT PORT	000000	RPn tied to Default Pin
U1TX	000001	RPn tied to UART1 Transmit
U1RTS	000010	RPn tied to UART1 Ready-to-Send
U2TX	000011	RPn tied to UART2 Transmit
U2RTS	000100	RPn tied to UART2 Ready-to-Send
SDO1	000101	RPn tied to SPI1 Data Output
SCK1OUT	000110	RPn tied to SPI1 Clock Output
SS1OUT	000111	RPn tied to SPI1 Slave Select
SDO2	001000	RPn tied to SPI2 Data Output
SCK2OUT	001001	RPn tied to SPI2 Clock Output
SS2OUT	001010	RPn tied to SPI2 Slave Select
CSDO	001011	RPn tied to DCI Data Output
CSCKOUT	001100	RPn tied to DCI Clock Output
COFSOUT	001101	RPn tied to CAN1 FSYNC Output
C1TX	001110	RPn tied to CAN1 Transmit
C2TX	001111	RPn tied to CAN2 Transmit
OC1	010000	RPn tied to Output Compare 1 Output
OC2	010001	RPn tied to Output Compare 2 Output
OC3	010010	RPn tied to Output Compare 3 Output
OC4	010011	RPn tied to Output Compare 4 Output
OC5	010100	RPn tied to Output Compare 5 Output
OC6	010101	RPn tied to Output Compare 6 Output

Peripheral Pin Select Registers (Output)

TABLE 4-37: PERIPHERAL PIN SELECT OUTPUT REGISTER MAP FOR dsPIC33EPXXXMU810/814 AND PIC24EPXXXGU810/814 DEVICES ONLY

File Name	Addr.	Bit 15	Bit 14	Bit 13	Bit 12	Bit 11	Bit 10	Bit 9	Bit 8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	All Resets
RPOR0	0680	—	—	RP65R<5:0>						—	—	RP64R<5:0>						0000
RPOR1	0682	—	—	RP67R<5:0>						—	—	RP66R<5:0>						0000
RPOR2	0684	—	—	RP69R<5:0>						—	—	RP68R<5:0>						0000
RPOR3	0686	—	—	RP71R<5:0>						—	—	RP70R<5:0>						0000
RPOR4	0688	—	—	RP80R<5:0>						—	—	RP79R<5:0>						0000
RPOR5	068A	—	—	RP84R<5:0>						—	—	RP82R<5:0>						0000
RPOR6	068C	—	—	RP87R<5:0>						—	—	RP85R<5:0>						0000
RPOR7	068E	—	—	RP97R<5:0>						—	—	RP96R<5:0>						0000
RPOR8	0690	—	—	RP99R<5:0>						—	—	RP98R<5:0>						0000
RPOR9	0692	—	—	RP101R<5:0>						—	—	RP100R<5:0>						0000
RPOR11	0696	—	—	RP108R<5:0>						—	—	RP104R<5:0>						0000
RPOR12	0698	—	—	RP112R<5:0>						—	—	RP109R<5:0>						0000
RPOR13	069A	—	—	RP118R<5:0>						—	—	RP113R<5:0>						0000
RPOR14	069C	—	—	RP125R<5:0>						—	—	RP120R<5:0>						0000
RPOR15	069E	—	—	RP127R<5:0>						—	—	RP126R<5:0>						0000

Legend: x = unknown value on Reset, — = unimplemented, read as '0'. Reset values are shown in hexadecimal.

Output Mapping

The thing to remember is

Remappable Output Pin = Functionality (Peripheral output)

So for example if we want U1TX on RP104

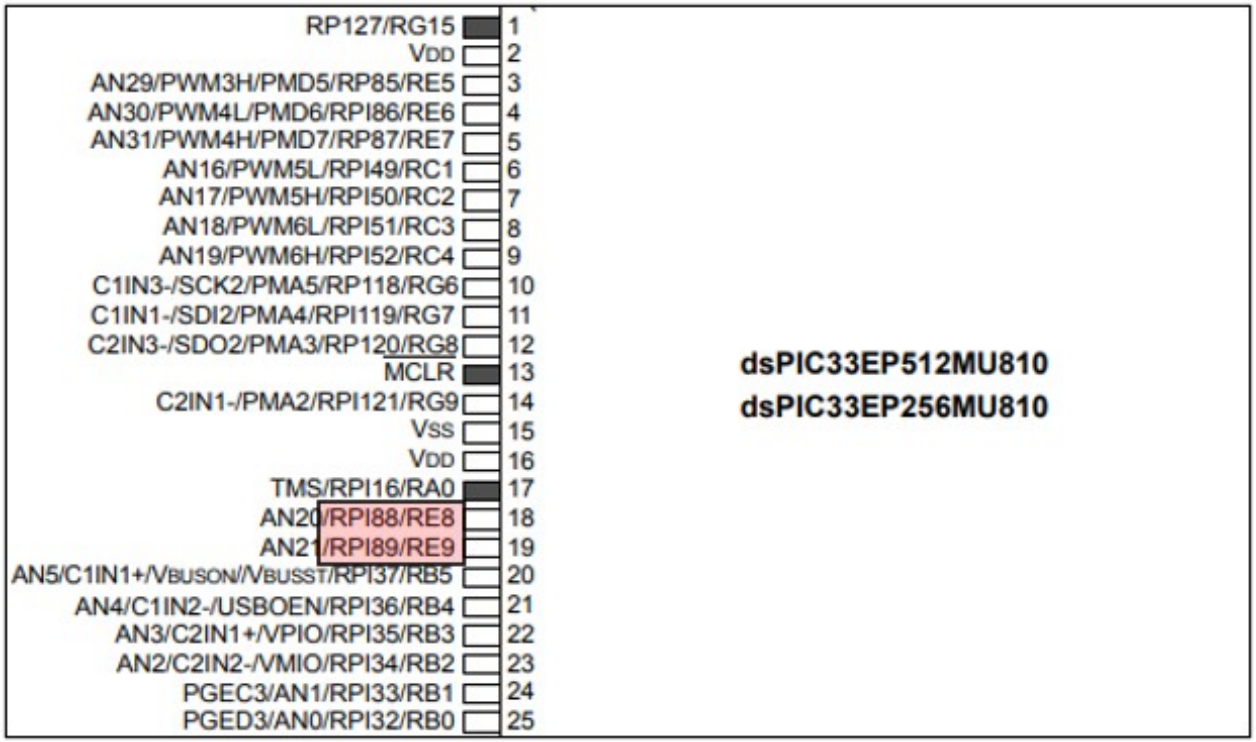
```
RPOR11bits.RP104R = 1; // 1 = U1TX
```

maps the U1RX to RP104. Instead, to map OC6 we can use one of the following equivalent statements

```
RPOR11bits.RP104R = 0b010101; // binary constant (value
directly reported on the datasheet)
RPOR11bits.RP104R = 21; // decimal equivalent
RPOR11bits.RP104R = 0x15; // hexadecimal equivalent,
often
preferred for compactness
```

Mapping Example

Therefore, to map button T2 (connected to RE8) to INT1, first find out which RPI is tied to the same pin as RE8



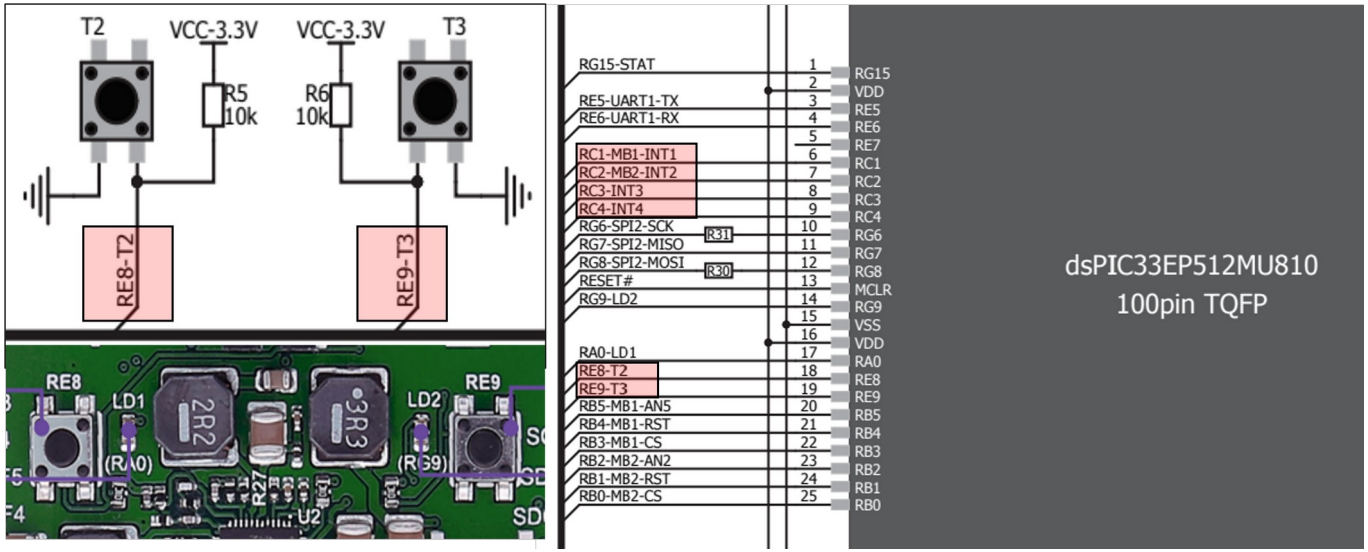
It is RPI88

Mapping Example

Example: Mapping INT1 to RE8 pin of left button

```
RPINR0bits.INT1R = 0x58; // 0x58 is 88 in hex
INTCON2bits.GIE = 1;     // set global interrupt enable
IFS1bits.INT1IF = 0;     // clear interrupt flag
IEC1bits.INT1IE = 1;     // enable interrupt
```

Assignment



1/ Blink LD1 led at 1 Hz frequency (500ms time on, 500ms off) without using interrupts; then make LD2 blink at 2 Hz frequency using interrupts.

2/ Blink LD1 led at 1 Hz frequency (500ms time on, 500ms off) without using interrupts; every time the button T2 is pressed, toggle the led LD2 using interrupts.



Università
di Genova